# Discord Community Analytics & Monitoring System

An End-to-End Discord Analytics Project - Version 1.0

*Anubhav Bharti,*
LinkedIn link, GitHub link

## Introduction

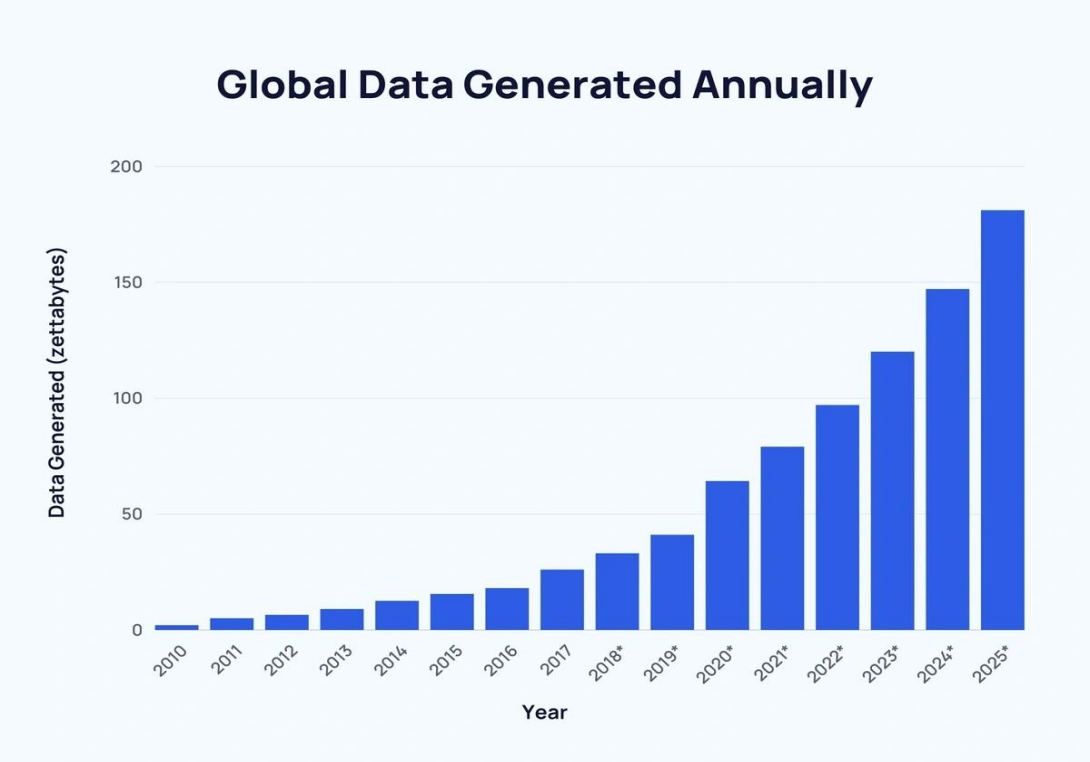
We all can't deny the fact that our world is overflowing with Data. From our smart watches to Hi-Tech missiles, from SPs to MNCs. In this digital era, data is the essence and the fuel running this new data first world. And it is growing at an exponential rate, especially after the dawn of AI.

---

**Around 90% of the world's data has been generated in just past 2 years.**

---

Yes! In just 2 years. From 120 zettabytes in 2023 to over 181 zettabytes by the end of 2025.




### Data is the New Oil

Everyone wants to extract, refine and use this oil to the best of their capabilities. It can give us those valuable insights which can play very crucial part in our decision making and help in achieving our objects. And so, I wanted to use this Oil too, for my own discord server.

# Table of Contents


Introduction ..... 1

Project Overview ..... 3

About Discord ..... 3

What is a Discord Server? ..... 4

What are Roles? ..... 4

My Discord Server ..... 4

Objectives ..... 4

Planning ..... 5

Data Scrapping ..... 5

Data Storage ..... 5

Data Visualization ..... 5

Workflow ..... 6

**Phase 1:** Creating Pipeline ..... 7

How Discord API works? ..... 7

What information to extract? ..... 7

Creating Discord Bot: Step by Step Guide ..... 8

Creating Database in PostgreSQL ..... 13

Initial Python Script ..... 14

Connect Power B.I. to PostgreSQL Database ..... 14

**Phase 2:** Final Python Script & Automation ..... 14

Key Features of the Script: ..... 16

Automation ..... 16

Final Testing ..... 18

**Phase 3:** Power BI (ETL & Dashboard) ..... 18

Dashboard Structure Idea ..... 18

Power BI Data Preparation & Modelling ..... 19

Dashboards ..... 20

Insights ..... 23

Learnings ..... 24

Limitations ..... 25

Security Measures ..... 26

Future Scope & Opportunities ..... 26

Conclusion ..... 27

# Project Overview

This project is about making sense of the people and activity inside a Discord Community.

In simple terms, it's about collecting the information of everyone present in a Discord Server to figure out several trends and patterns about the Server.




It starts by,

- **Collecting** the relevant data members, like their username, discord id, roles, joined date etc.
- **Storing** all that data safely in a database.
- Converting the raw data into easy-to-understand **dynamic charts**.

So instead of looking through Discord by hand or guessing about community trends, this system allows me clear, up-to-date insights. Like, how many people have joined in a particular time frame, how the server is doing so far by looking at the growth trends, how many people are interested in different activities? And much more, which will essentially help the server admins like me, to have an **eagle's eye** on how the server is doing in general and to take **data driven decisions** for the server by looking at the trends & patterns.

Everything is set up to run automatically, so we don't have to do the work each time. Just one click (technically 2 because we have to click on the refresh button too) and we will get an up-to-date dashboard to figure out how the server is doing by just one look.

It is a powerful system that can help server owners and managers truly understand and manage their community really well.

In this blog, I'll share full **end-to-end pipeline** of this project and discuss the **problems**, **challenges**, **planning**, **solutions**, **features**, **limitations** and **future scope** too.

## About Discord

In a world where online communities thrive, Discord has emerged as one of the most popular platforms for connecting people effortlessly. Whether you're a gamer, coordinating strategies, a professional team collaborating on projects, or a hobbyist sharing interests, Discord offers an **interactive space** where conversations flow smoothly and in an **organizable manner**.

It's not just a free voice, text and video communication app, it's much more than that. Users can join or create servers, which act as dedicated spaces for different topics and activities. Within those servers, there are channels & threads where members can discuss specific topics, share ideas and engage with each other.

Some of the features are:

- Instantly **send messages** or jump into **voice calls** with friends and colleagues.
- Host **video meetings**, stream **gameplay**, share **presentations** effortlessly, listen **music**, watch **YouTube**, play **discord games** and have fun together.
- Customizable **Roles & Permissions** for better control and management on the server.
- Available on Windows, macOS, Linus, iOS, Android and web browsers.

## What is a Discord Server?

It is like our own digital community-built around a shared interest. Inside a server, we have 2 types of channels:

1. Text Channels (for chatting, sharing links & discussions)
2. Voice Channels (for live conversations or gaming sessions)

## What are Roles?

To manage the crowd, Discord uses roles-labels with specific permissions. Roles decide who can access certain channels, manage settings or moderate the server. But not only that, they also work as labels, denoting the identity or interest of the member who has them.

I think this is the right time to introduce you to our own little server this project is based on. I'll explain the roles and how crucial and important they can be with the help of my own sever.

## My Discord Server

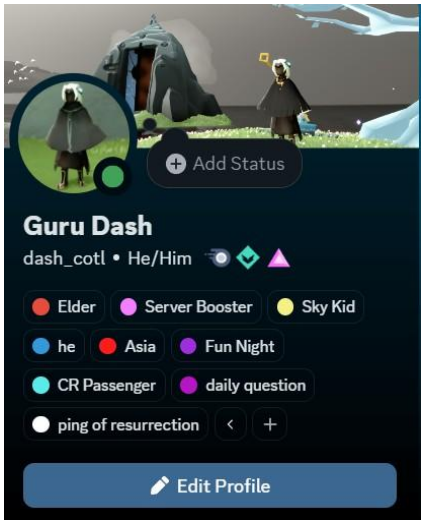
Me and my friends run a server called Sky Kids: Heads Up! – friendly space for fans of the game Sky: Children of the Light We hang out, share game tips, arts, pictures, organize events and just chill.

In our server, we have 3 types of main hierarchical roles:

1. **Elder**: For the Server Owners. (Me & my partner Kiwi) – Full Control.
2. **Ancestor**: For Moderators & Event hosts – Trusted, but limited in server settings.
3. **Sky Kids**: All of the members – Permissions to let them chat, join events and enjoy the community

**Activity-Based Roles**: Members can choose roles based on interests (like Fun Nights, Contests, Giveaways, etc.) to get notifications only for what they care about.

These Roles are the real backbone of this whole project.




*1 A snippet of how roles look like.*

# Objectives

Imagine you're the admin of a bustling Discord community. At first, things are simple-just a handful of friends chatting and sharing memes. But as your server grows, so do your headache.

As an admin, I often wondered how my server was really doing-not just by the total number of members, but by understanding the community's pulse. Discord offers some basic stats, but they're limited-and smaller servers like mine often don't even have access to those. I needed dynamic, customized insights tailored to my server, to solve these challenges:

- **See beyond simple numbers**: Understand real trends in member activity and growth.
- **Know my community better**: Learn about member diversity and engagement.
- **Measuring Impact of Events**: Assess how server events or announcements affect member engagement.
- **Track retention**: See not just who joins, but who stays and figure out the possible reasons.
- **Overall Growth**: Figure out Weekly, Monthly & Overall growth of the server.

- **Managing Everything Efficiently**: Avoid the hassle of scattered data and endless manual tracking.
- **Reducing Guesswork**: Make decisions based on data, not assumptions or gut feelings.
- **Saving Time**: Automate repetitive tasks so admins can focus on community building instead of data wrangling.

In short, I needed a **smarter**, easier way to truly understand and manage my Discord community-**something far beyond what Discord's tools could offer**.

The objective was clear:

**To create an interactive, dynamic dashboard that provides a comprehensive overview of server activity and extracts as much useful information as possible.**

# Planning

To tackle these challenges head on, I knew I needed a clear plan and a structured approach. So, I began mapping out exactly how this project would work.

Key questions were:

- What information can be extracted from Discord, and how?
- How can this data be stored efficiently and securely?
- What's the best way to access and visualize the information quickly?
- How will the entire pipeline operate?
- What security measures are necessary to protect the data?

After extensive research, contemplation, and several design iterations, I finalized the following approach: (It's time to get a bit technical)

## Data Scrapping

A custom **Discord bot** built with **Python** is at the core of the system. This bot can extract a wide range of server data, including: Member's usernames, nicknames, Discord IDs, avatar URLs, server join dates, current status, message history, and more. This opens up endless possibilities for analysis and insights.

## Data Storage

All scraped data is stored in structured tables within a **PostgreSQL RDBMS**. **Why PostgreSQL?**

- I'm familiar with it, making development and troubleshooting easier.
- It's simple, reliable and scalable for large datasets.
- And it is underrated, the modern UI with black theme looks cool. *Lol*

## Data Visualization

**Power BI** was chosen for visualization because:

- It has a native PostgreSQL connector for seamless integration.
- Supports DirectQuery for real-time data access.
- Offers advanced data transformation (Power Query) and analytics (DAX formulas).
- Handles large datasets efficiently and produces interactive dashboards.

## Workflow

The initial mind map for the pipeline:

**Python** runs (the **Discord bot** to **scrape data** → **Saves data** in **PostgreSQL** database) → **Power BI** connects for ETL & **Visualization**.

To make it all fully automated I needed answers to some questions:

- **Should the script run through online cloud services or offline on my local machine?**

I chose to run the script locally (on my own machine) rather than using cloud services because:

- **Better Security:** Sensitive data like API tokens stay on my machine, reducing exposure to breaches.
- **No Reliance on Cloud Uptime:** Free cloud services can be unreliable or have strict limits.
- **Lower Complexity & Cost:** No need to configure or maintain the cloud environments.
- **Compliance and Privacy:** Running locally minimizes risks related to Discord's terms and private server data.

- **How to execute the Script Run?**

After deciding on local execution, I considered three options:

- **Run Continuously:** Always updated and more accurate data but wastes resources (CPU & Memory), requires manual management and risks overheating or slowing down the machine.
- **Scheduled Runs:** Automates updates but depends on machine uptime and can waste resources when unnecessary.
- **Manual Runs via Batch File:** Simple, secure, and resources-efficient. It'll run the script only when I want fresh data.
- I chose **manual execution via batch files** because it offers the best balance of control, security, and efficiency for my needs.

To streamline the workflow and reduce repetitive manual steps, I chose to create **Nested batch files.**

First, I created a batch file which-with just one click, will execute the original python script to extract data and save it in database with the help of python.

For even greater convenience, I decided to create another batch file which will first run the above-mentioned batch file and then open the specific Power BI dashboard.

Why I chose to do it instead of running everything in just one batch file?

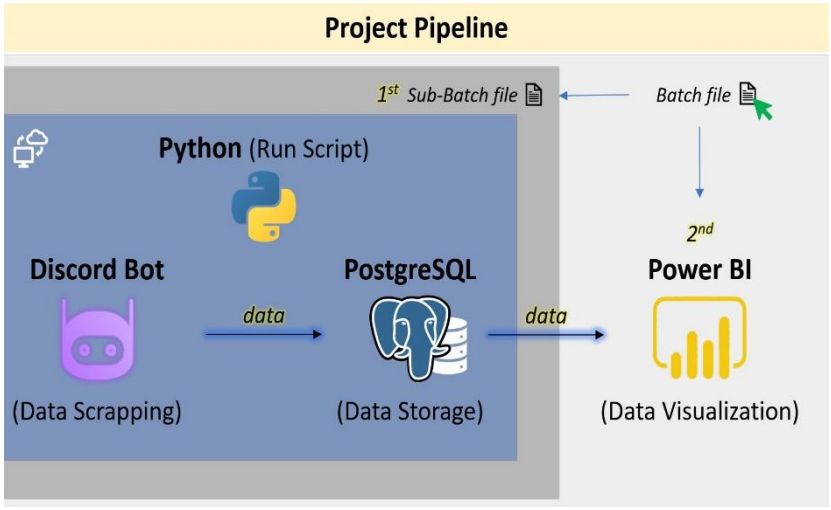
Because first, we need to make sure that data is updated safely in the database and then we can open the Power BI dashboard.

This chained automation means I can update and analyze my Discord data with just one click. No command-line work or manual setup required.

I divided the execution of this plan in 3 Phases:

- **Phase 1:** Build the Python Discord bot, set up PostgreSQL, and connect to Power BI.
- **Phase 2:** Finalize the Python script and automate the workflow with batch files.
- **Phase 3:** Perform ETL and advanced analysis and build Dashboard in Power BI.

*Final Pipeline Workflow*




# Phase 1: Creating Pipeline

Before start building the discord bot, we need to understand a bit about how discord API works and what information do we need.

## How Discord API works?

Discord's API operates through HTTP requests and WebSocket:

- **REST API**: Used for stands operations like sending messages, managing users, and configuring servers.
- **WebSocket API**: Used for real-time events like message updates, user status changes, and reactions.

To interact with Discord's API, developers need an API token:

- **Bot Token**: Used for Bot authentication.
- **OAuth2 Token**: Used for User authentication in third-party apps.

## What information to extract?

It's pretty clear that we can extract almost every kind of information. But just because we can, doesn't mean we should. We only need to extract the information which will help achieve our objectives while keeping things effective and efficient.

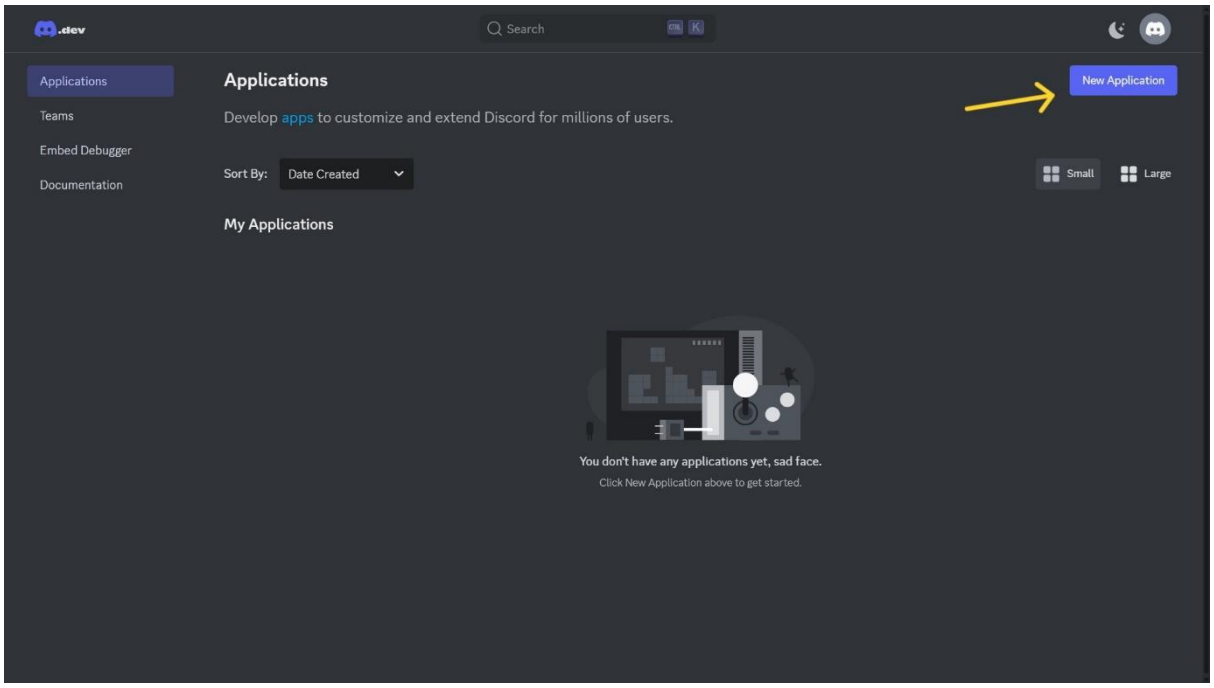
After some contemplation, I decided to extract the following information:

- **Discord ID**: A unique identifies for each user on Discord. It will serve as a reliable primary key for tracking users across the databases and avoiding duplicates.
- **Username**: The user's global Discord username. Useful for identification, reporting, and referencing users in dashboards.
- **Display Name**: The user's nickname on the specific server (may differ from their global username). Reflects how members are seen within the community, which can be important for identification and moderation.
- **Top Role**: The highest-ranking role assigned to the user. Indicates the user's authority within the server. (e.g., admin, moderator, regular member).

- **Roles**: All roles assigned to the user, as a comma-separated list. Shows the user's group affiliations within the server and activities they are interested in.
- **Status**: The user's current online status (online, idle, dnd, offline). Can help understand availability of members in real time.
- **Server Join Date**: The date and time the user joined the server. Useful for tracking member retention and growth and onboarding trends.
- **Role Count**: The total number of roles a user has. Quickly identifies highly-involved or multi-faceted members.
- **Boosting Since**: The date the user started boosting the server, if applicable. Allows recognition of supporters and analysis of boosting trends.
- **Account Creation Date**: The date the user created their Discord account. Helps distinguish between new and experienced Discord users, and can aid in detecting alt or spam accounts and also has the potential to be one of the criteria to find moderators for the server.
- **Guild Permissions**: The user's permissions within the server. Important for security audits and understanding who can manage or moderate the server.
- **Pending Status**: Whether the user has completed server membership screening. Identifies new members who haven't yet passed onboarding requirements, which can be useful for moderation and follow-up.

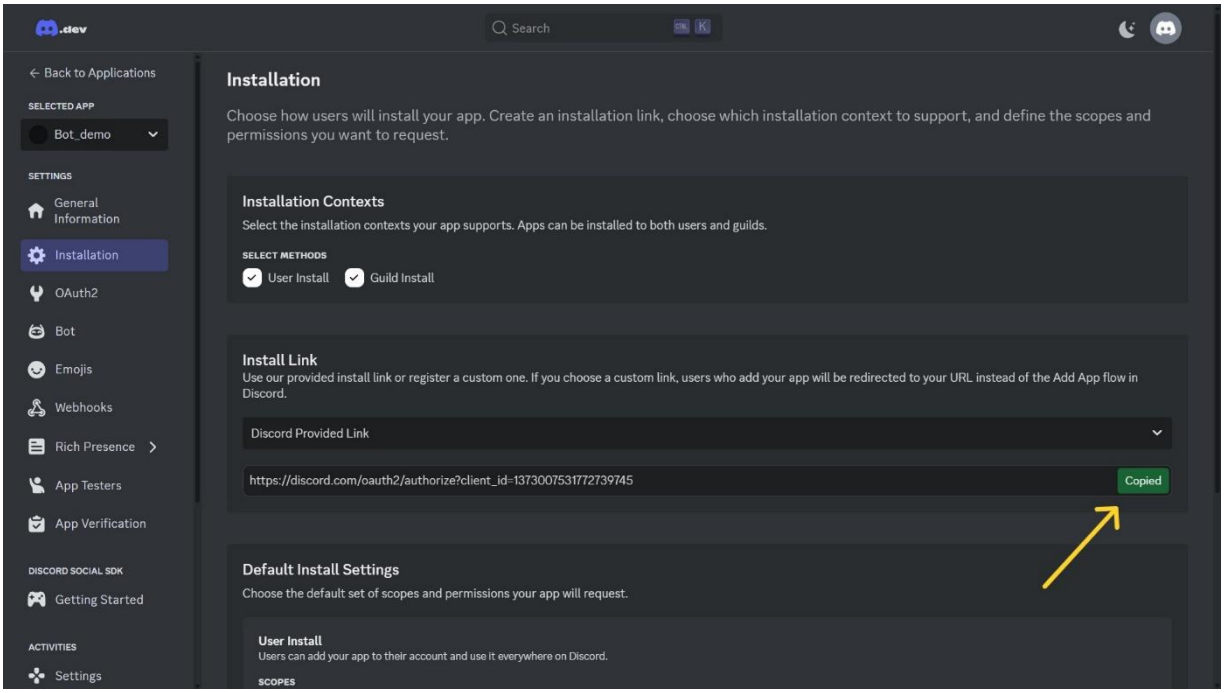
## Creating Discord Bot: Step by Step Guide

1. Go to the Discord Developer Portal and login with your account.
2. Click on New Application, give it a name and write a description about it (optional).



3. Navigate to the ‘Installation’ section

    a. Copy either the Discord provided install link or choose a Custom URL. (This is to redirect the user who add the app to this link. In this case, I continued with Discord provided install link)

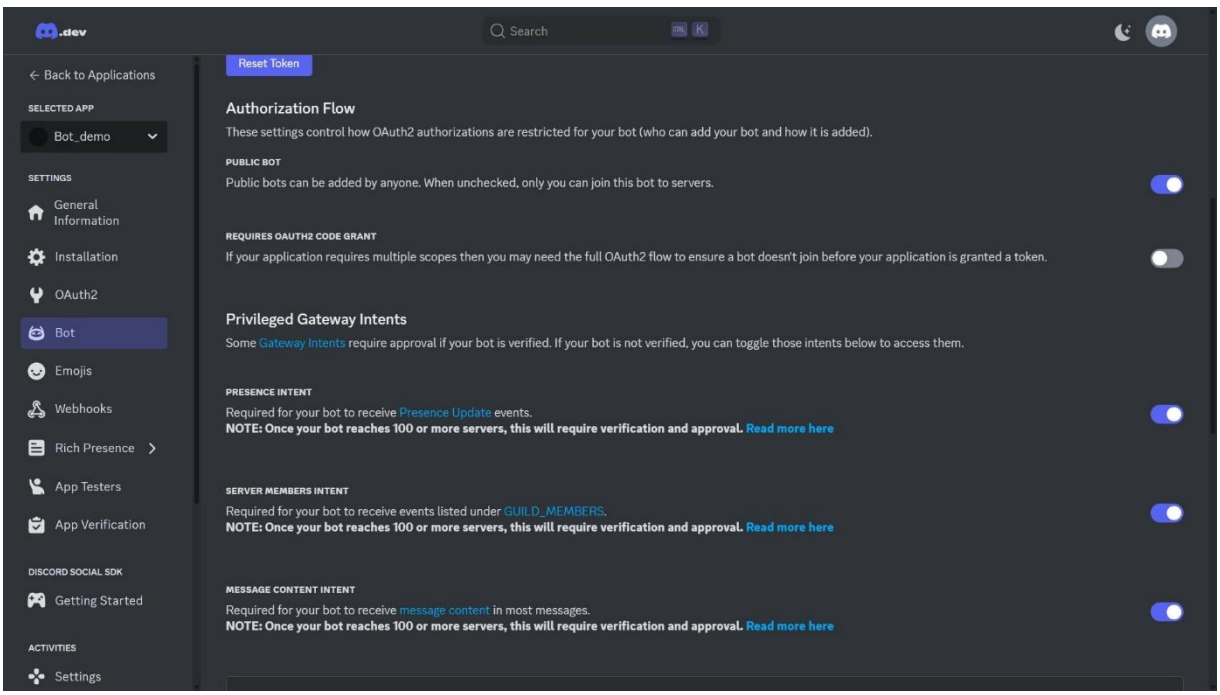



4. Navigate to ‘Bot’ section

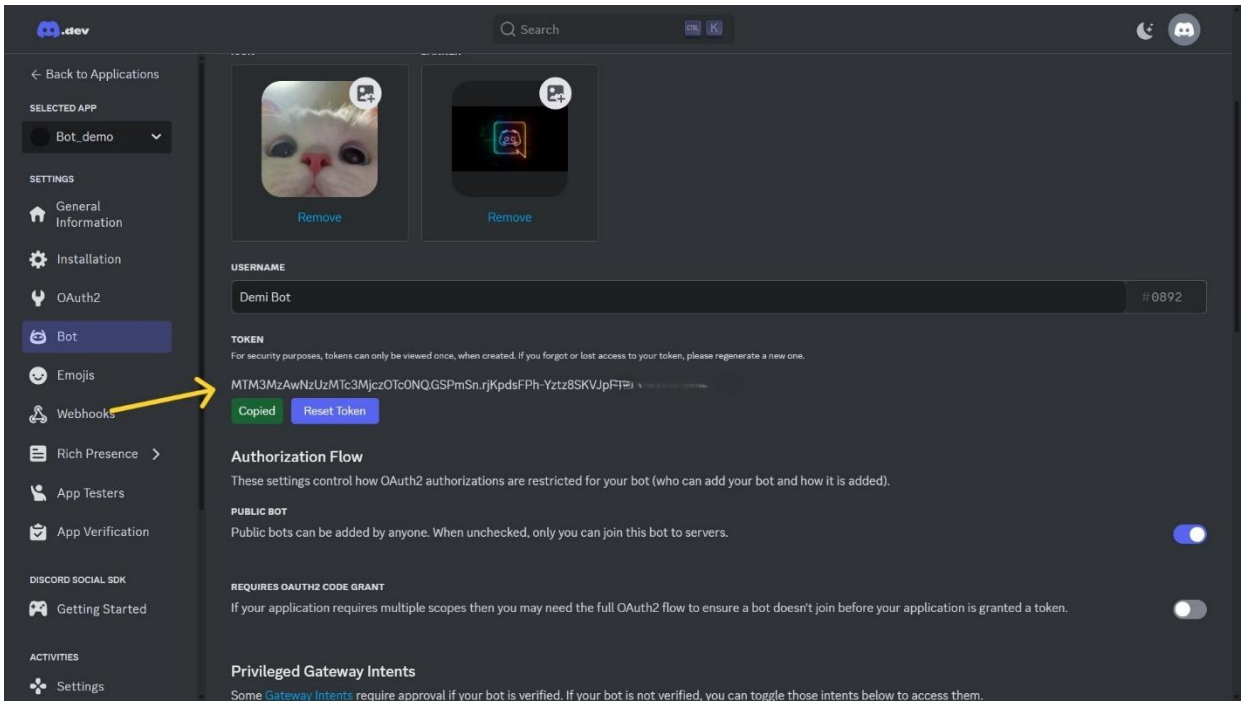
    a. Select a nice Icon and Banner for your Bot.

    b. Give it a username.

    c. Select Authorization Flow as follows:

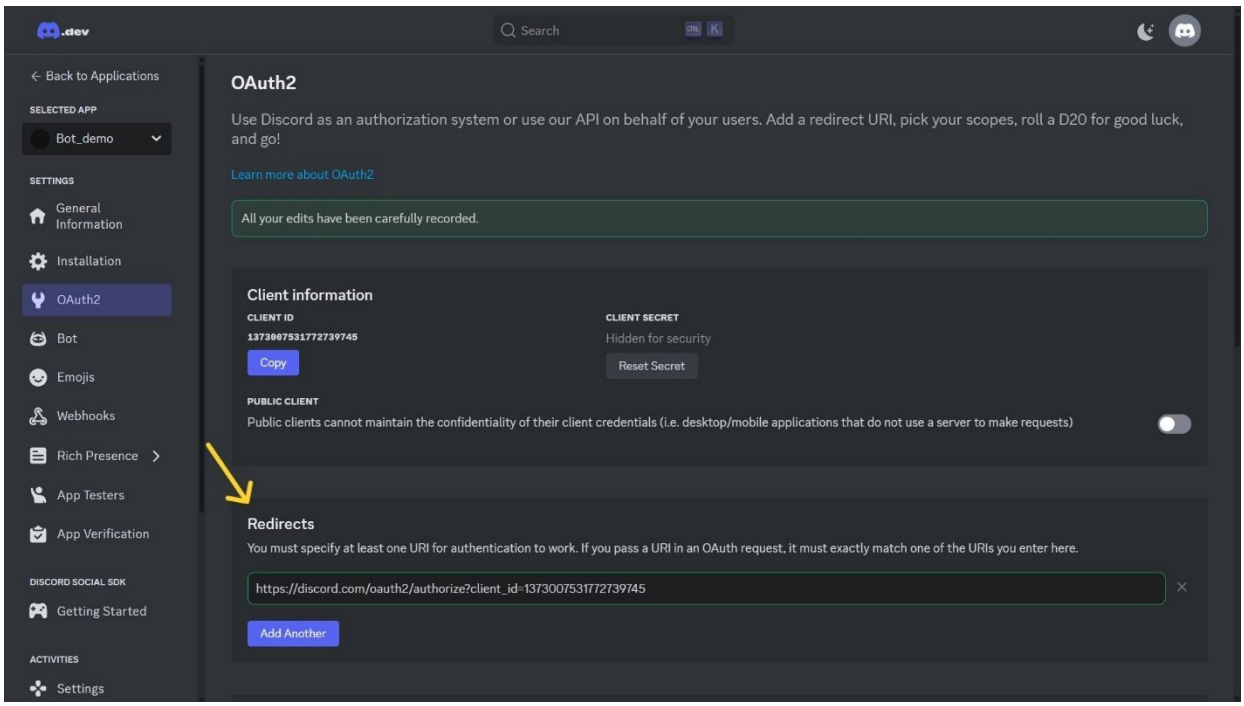


d. Click on ‘Reset Token’, pass the credentials and copy and paste the newly generated token in a safe and secure place for later use.

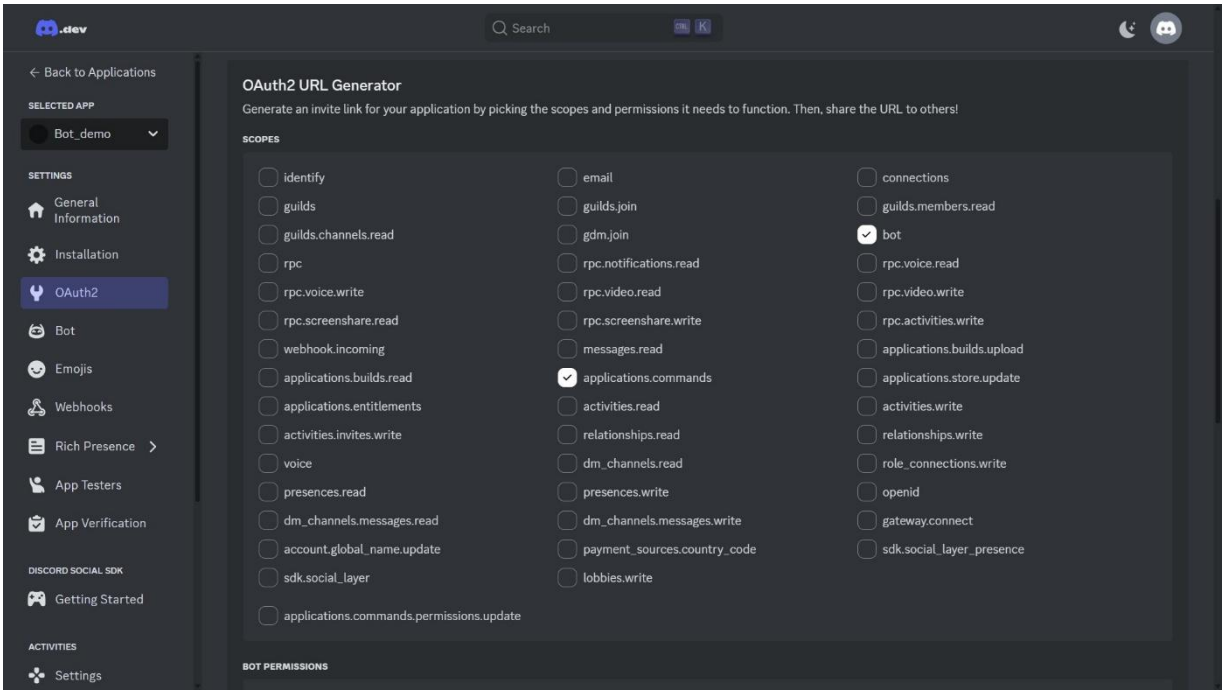



5. Navigate to the ‘OAuth2’ section

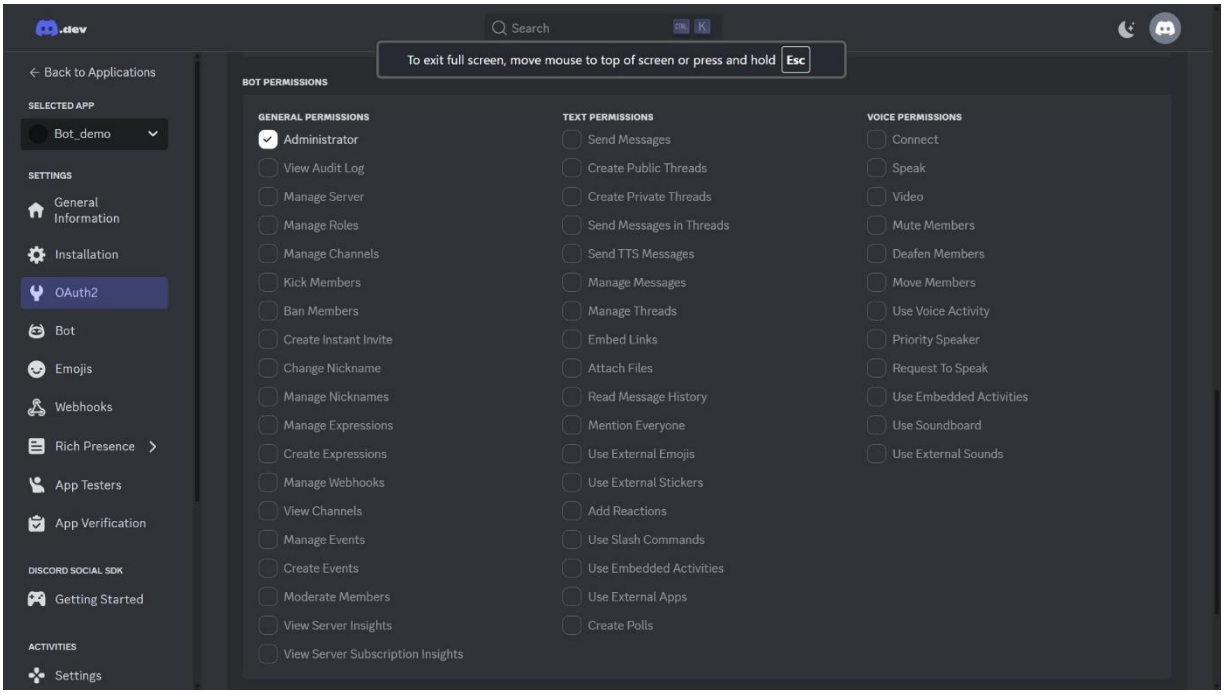
a. Paste the install link copied earlier, in the ‘Redirects’ options.



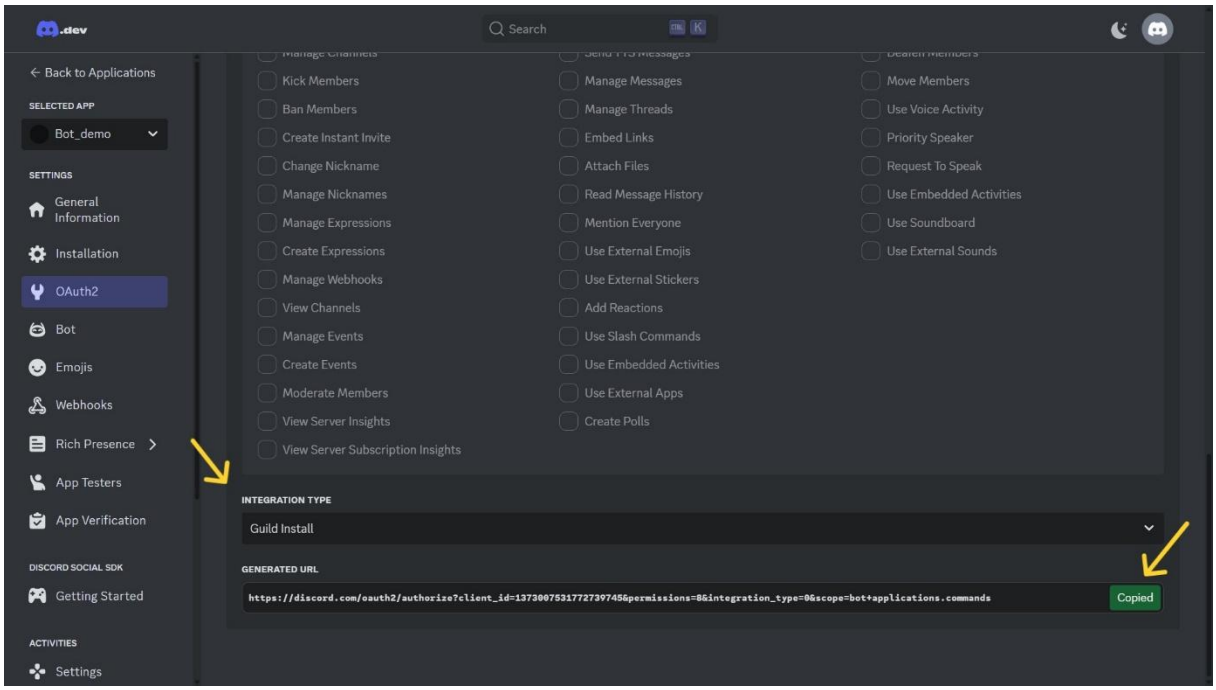
b. Go to ‘OAuth2 URL Generator’ section below it and pick the scopes and permissions it needs to function. In our case, selecting only ‘bot’ scope should be enough but in case if we need to upgrade the bot in the future and use slash commands in the future we need to add ‘application.command’ scope as well.



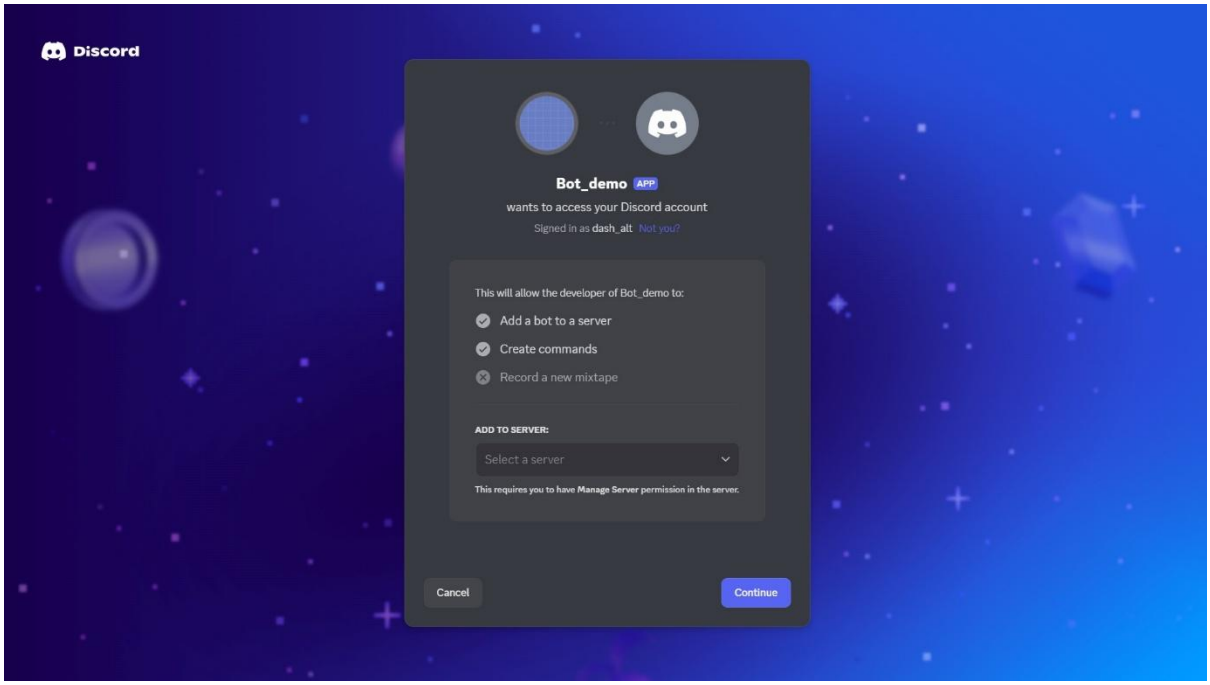

c. Got to the next ‘Bot Permissions’ section below. The permissions you select depend on what your bot needs to do. For a typical data extraction and management bot, ‘Read Message’, ‘View Channels’, ‘Read Message History’, ‘View Members’ permissions should be enough. You can also give ‘**Administrator**’ permission but only if you trust your bot and want to simplify setup; this overrides other permissions. As this is our bot, we can safely give it ‘Administrator’ permission.



d. Select the integration type: Select ‘Guild Install’ to install it for server.




6. Copy and paste the generated URL in the URL section of a separate webpage.




7. Select the server you want to install it to and continue until the bot gets integrated with the server and done! You have successfully created a discord bot. Click here to know more

## Creating Database in PostgreSQL

Before creating the database, I had to address following concerns:

- **How to keep track of members who leave the server to analyze metrics like retention rate and churn rate?**

To achieve this objective, we need to save their data in our database. 'Discord ID' will help us to track members who leave the server. What we can do is,

Whenever we run the python script, it will compare the discord ids which are already present in the database to the freshly fetched discord ids.

If any discord id which is present in the database but not present in the newly fetched discord ids, means the member has left the server and it will automatically save the details of that specific id in a separate table with current datetime so we will know exactly before what datetime the member left the server.

We can call the table as 'former members' and the other column as 'left before'.

- **How to make sure that the datetime is always consistent and according to GMT 00:00?**

To address this concern, we need to explicitly convert all timestamps into UTC before storing or processing them. This can be done using 'datetime' module in python. We also need to make sure that the 'left before' column datetime which will be assigned according to the current datetime of when the script will run, should be in the same format.

- **How to design the structure of database in the most effective manner to keep track of current members & former members? while also addressing the fact that the script won't be running continuously but manually.**

After considering many possibilities I came up with the following plan:

The data will be stored in 3 table:

1. **Current Members**: To store the data of Members which are current present in the server. Discord Id column will be used to uniquely identify each row.
2. **Former Members**: To store the data of Former members. Discord Id column will be used to uniquely identify each row.
3. **Members Events**: To easily track the events of members who leave and join the server. This will record an entry-each time a member joins or leave the server. Even if it's the same member doing it multiple times. A unique event id will be automatically assigned to each entry according to event's date and time, and will be used to uniquely identify each row (Event Datetime is the datetime when a member joins or leave the server).

Whenever the script will run, it will compare the discord ids present in current members table to the newly fetched discord ids.

In case the id isn't present within the already existing members table, means it is a new member, and the details will be fetched and saved in current members table also updating the member events table.

In case the id isn't present within the newly fetched discord ids anymore, means the member has left the server, and the details will of that specific id will be transferred from current members table to former

members assigning the current datetime according to GMT 00:00 and will also update the member events table accordingly.

To increase the efficiency, we can use **UPSERT** for conflict value handling and don't include any unnecessary transformation to the data which will keep the updating to minimal, essentially making the process much faster.

### Initial Python Script

After making sure our database is working as intended, it was time to work on python script to **connect our bot** and **save the data in database**.

1. Open IDE for python and create a new project. (For me, its Pycharm)
2. Create a new .py file.
3. Import 'discord.py' library to use discord-based commands and started building the script.

At this point, I started with a basic script that would scrap the data and save it in the database with hard coded credentials. The purpose was to debug and making sure everything is working as intended. The Final Script was prepared in Phase 2.

***Pro Tip**: It's better to create a separate python file for testing purpose only and later add the changes in your original script. This will prevent unwanted changes into the original script.*

### Connect Power B.I. to PostgreSQL Database

1. Open a new Power B.I. file.
2. Navigate to **Get Data**
3. Select PostgreSQL as the data source and enter credentials for the database.
4. Make sure to cross-check if it's returning the correct and complete dataset.

After running several tests of core pipeline (from the fetching data to pushing it in Power BI) and making sure that everything is working as it should. Time to commence Phase 2.

## Phase 2: Final Python Script & Automation

### Final Python Scripting:

I gradually improved the script, resulting in an effective and efficient code that can fetch and save the data in my database while making sure that every concern is addressed properly. Find the full code in my GitHub Repository.

### Script Workflow:

1. **Setup & Initialization**

- **Secure Configuration:** Loads Discord bot tokens and PostgreSQL credentials from a **.env** file, avoiding hard-coded secrets.
- **Logging:** Configures logging to track every step (info, warnings, errors) for traceability.

2. **Connecting to Discord & PostgreSQL**

- **Discord Bot Initialization:** Uses **discord.py** with required intents (**members**, **presences**) to access member data.
- **Retry Logic for Guild Fetching:** If the target server isn't immediately found (e.g., due to API delays), the script retries multiple times.
- **Database Connection:** Establishes a PostgreSQL connection with error handling to prevent crashes on startup.

3. **Synchronization Process**

- **Chunked Member Fetching:** For large servers, members are retrieved in batches (pagination) to avoid API rate limits and memory overload.
- **Set-Based Comparison:**
  - Compares in-memory Discord member IDs with database records using Python sets.
  - Identifies **new members**, **existing members**, and **left members.**
- **Batch Processing:**
  - **New Members:** Inserted in chunks (e.g., 500 at a time) using **INSERT ... ON CONFLICT DO UPDATE** to avoid duplicates and logged in member event table.
  - **Existing Members:** Updated in bulk with latest roles, statuses, and permissions.
  - **Left Members:** Archived into a **former members** table and logged in member event table.

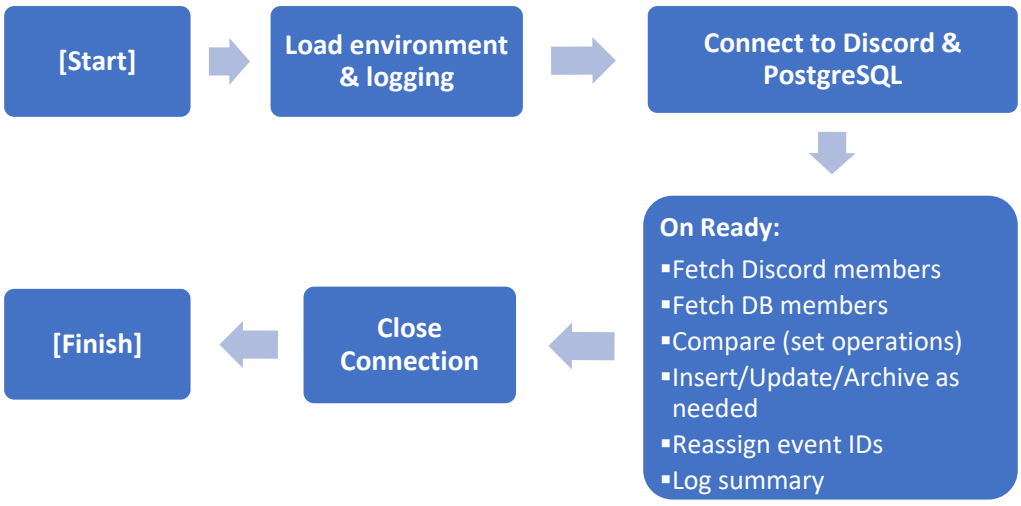
4. **Database Operations**

- **Data Validation:** Ensures mandatory fields (e.g., **discord_id**, **username**) are present before insertion.
- **Event Logging:** Every join/leave is recorded in **member_events** with UTC timestamps.
- **Event ID Reassignment:** Maintains chronological order of events, even after rejoin/leave cycles.

5. **Error Handling & Recovery**

- **Graceful Failures:** Uses **try-except-finally** blocks to catch errors, log them, and roll back incomplete transactions.
- **Async/Await:** Non-blocking I/O keeps the bot responsive during large syncs.

- **Controlled Shutdown**
- **Clean Exit:** Closes database connections and stops the bot gracefully with **await bot.close()**, even if errors occur.



## Key Features of the Script:

1. **Data Integrity**

- **Conflict Resolution: ON CONFLICT DO UPDATE** ensures no duplicate entries.
- **UTC Timestamps:** All events use UTC for consistency.
- **Validation:** Mandatory fields checked before database writes.

2. **Reliability**

- **Retry Logic:** Automatically retries guild fetching on transient API failures.
- **Atomic Transactions:** All database changes are batched and committed together.
- **Archiving:** Departed members are preserved in **former members** for historical analysis.

3. **Performance**

- **Async Operations:** Concurrent I/O handling (Discord + PostgreSQL) reduces total runtime.
- **Chunked Processing:** Fetches/writes members in batches to avoid memory/API limits.
- **Set Logic:** Instant member comparisons using Python's optimized sets.

4. **Security**

- **Least Privilege:** Bot uses minimal Discord intents and database permissions.
- **Parameterized Queries:** Prevents SQL injection attacks.
- **Environment Variables:** Sensitive data stored in '**.env'** file, excluded from version control.

This script is designed for **scalability** and **low overhead**, making it ideal for large servers.

- **Speed:** Async operations and batch processing cut sync times by 30–50%.
- **Accuracy:** Conflict resolution and validation ensure clean data.
- **Resilience:** Retries and error handling keep the bot running smoothly.
- **Security:** Credential isolation and parameterized queries block common attack vectors.

By balancing these elements, the script delivers a robust, enterprise-grade solution for Discord community analytics.

Now that we are done with our python script (the hardest part). Let's move on and make things automatic with minimal manual intervention.

# Automation

The goal is to update the database and launch the Power BI dashboard without any manual steps or command-line hassle. Using Batch Files.

**What is Batch File?** It is a type of script file used in Windows operating systems which contains a series of commands that are executed in sequence by the command-line, configure system settings, and more. It's like creating a to-do list for your computer to follow automatically. It is saved with .bat extension.

We'll use simple Windows batch files to:

1. Run your Python script automatically.
2. Open your Power BI dashboard as soon as the data is refreshed.
3. (Optionally) Place a shortcut on your desktop for true one-click access.

Step 1: **Create a Batch File to run the Python Script:**

- Open Notepad and write the following command:

```
@echo off
"C:\Users\YouName\Path\To\your\PythonInterpreter\python.exe" "C:\Users\YouName\Path\To\Your\PythonScript.py"
```

**@echo off** command hides the commands in the batch file from being displayed on the command prompt, except for their output. So, in this case, we will only look at our data logs, essentially providing us a user-friendly experience.

- The first path is your Python interpreter.
- The second path is your Python script.

- Save it in .bat extension in the same folder as your script.
- Double-clicking this file will now run your Discord data extraction script

Step 2: **Create a Wrapper Batch File for Full Automation**

- Open Notepad and enter following:

```
@echo off
REM Run the Python script batch file
call "C:\Path\To\Your\run_script.bat"
REM Wait for the script to complete (adjust if needed)
timeout /t 3 /nobreak
REM Open the Power BI dashboard file
start "" "C:\Path\To\Your\dashboard.pbix"
```

- Adjust the timeout if your script takes longer to finish.
- Save this as .bat file with your desired name.

Step 3: **(Optional) Create a Desktop Shortcut**

- Right-click your wrapper batch file and select **Create Shortcut**.
- Place the shortcut on your desktop for easy, one-click access.

**Result:** Now, single-clicking your shortcut will:

- Run your Discord member extraction script.
- Wait for it to finish.
- Open your Power BI dashboard with the latest data.

**Benefits of This Approach:**

- **Saves Time:** No manual steps or technical commands-just double-click and go.
- **Reduces Errors:** Automation ensures the same process runs every time, minimizing human mistakes.
- **User-Friendly:** Anyone can update the dashboard, regardless of technical skill.
- **Scalable:** Works for large Discord servers and complex Power BI dashboards.

## Final Testing

Once the automation is set up, it's important to verify that the process is working as intended and that the Power BI dashboard is reflecting the latest data.

**How to Test?**

1. **Run the Automation**

- Click the desktop shortcut or batch file.
- Observe the command prompt window (if visible) for any errors or logs.

2. **Check the Database**

- Before and after running the batch file, inspect the database (using a tool like pgAdmin) to confirm that new Discord member data has been inserted or updated.

3. **Open Power BI**

- When dashboard opens, check that the visuals and tables reflect the latest member data.
- Use the **Refresh** button in Power BI to see the latest changes and observe if the changes are correct.

4. **Troubleshooting**

- In case of missing or outdated data, check the logs/output from the script for errors.
- Ensure the batch file paths are correct and the Python script completes without any issues.

# Phase 3: Power BI (ETL & Dashboard)

The Dashboard need to answer following questions:

- How is the Discord server performing overall?
- What is the growth, retention, and churn rate?
- What are the details of server members that I can leverage?
- How can I manage and grow the community more efficiently?
- What data-driver decisions can I make to improve the server?

To answer these, I needed a dashboard that:

- Gives a quick health check of the server.
- Shows growth trends and patterns.
- Provides detailed breakdown of current and former members.
- Surfaces actionable insights for server management.

## Dashboard Structure Idea

After careful consideration, the dashboards were divided into 4 pages:

1. **Server Status:** A quick overview of the server's current state and recent performance.
2. **Growth Insights:** Track long-term and recent growth trends, and see how the server evolves over time.
3. **Member Insights:** Deep dive into the current member base: who they are, what they do, and how they interact.

4. **Former Member Insights:** Analyze members who have left to spot patterns and potential issues.

## Power BI Data Preparation & Modelling

### Data Import & Cleaning:

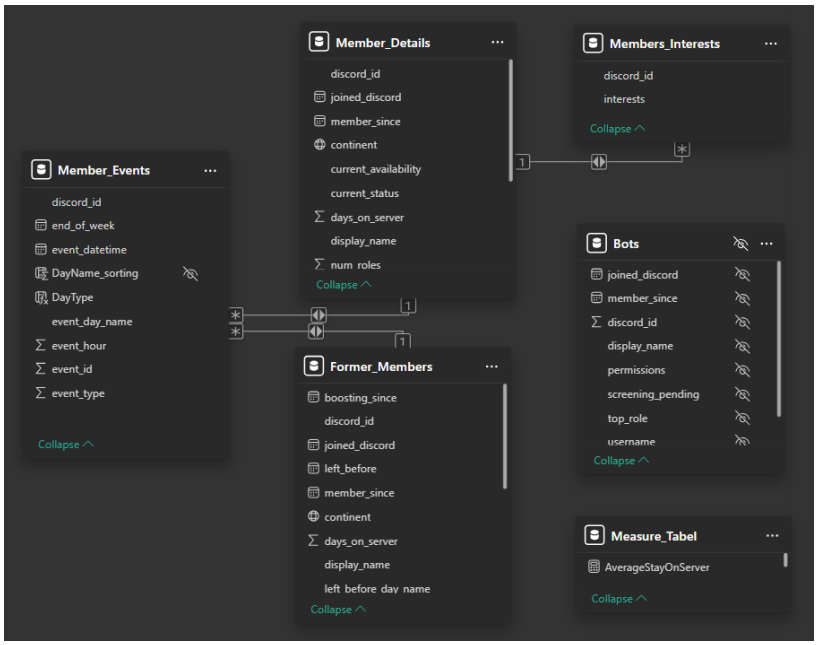
- Source: Import 3 main tables from the database: Server_Members, Former_Members, Member_Events.
- Power Query Steps:
  - Separate Bots: Filtered out bots into their own table.
  - Cleaned Columns: Split roles, activities, pronouns, continents into separate columns/tables for easy analysis.
  - Created Relationships: Linked tables by discord id for member-level analysis.
  - Calculated Columns: Derived fields like days on server, server age category, booster status, etc.

LINK TO POWER QUERY TABLES

### Data Model:

Measures like Growth %, Total Joining, Net Joining, Retention Rate, Churn Rate, Average Stay, Weekly Joining Rate, etc. were calculated using DAX, leveraging the cleaned and related table for accuracy.

Only those tables are included in the model which are essential for calculations and visualizations:




**Member_Events**: For event-based analysis which is connected to both, Former_Members table and Member_Details table.

**Former_Members**: To Analyze former members.

**Member_details**: To Analyze current members.

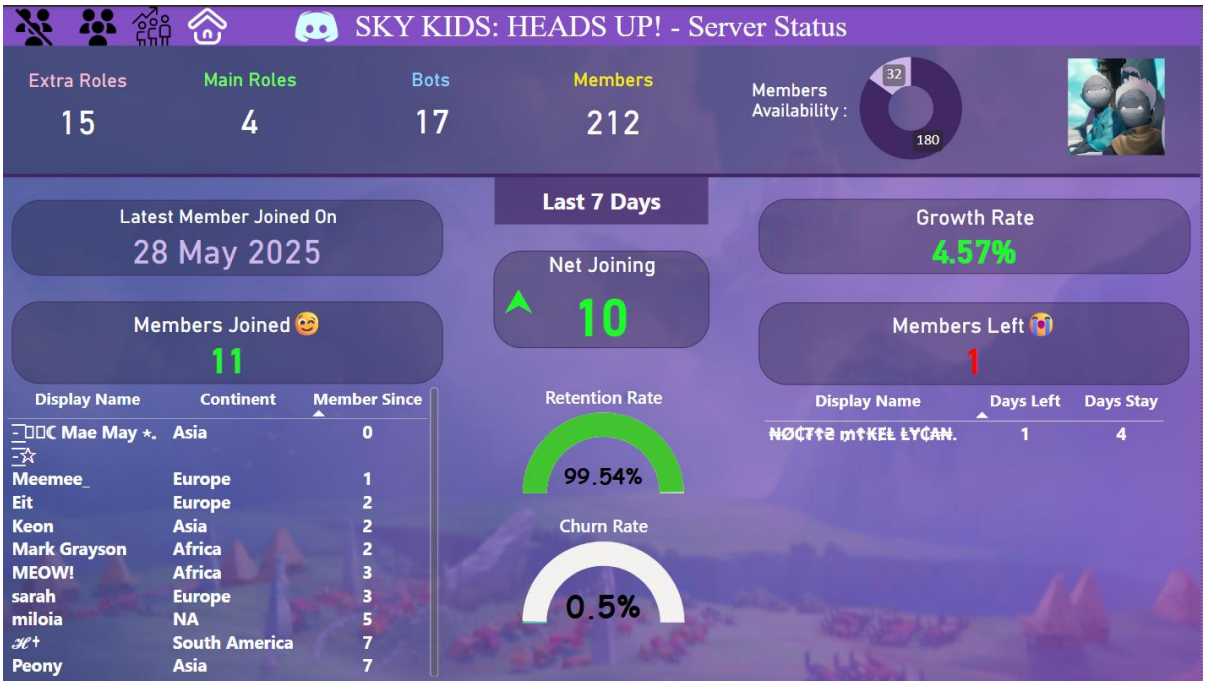
**Member_Interests**: To Analyze different interests of members.

**Bots**: To show the number of bots we have. Yes, That's it.

**Measure_Table**: Contains all the measures for our dashboard.

## Dashboards

**Server Status:** Are we growing or shrinking this week? Who Joined & Who Left?

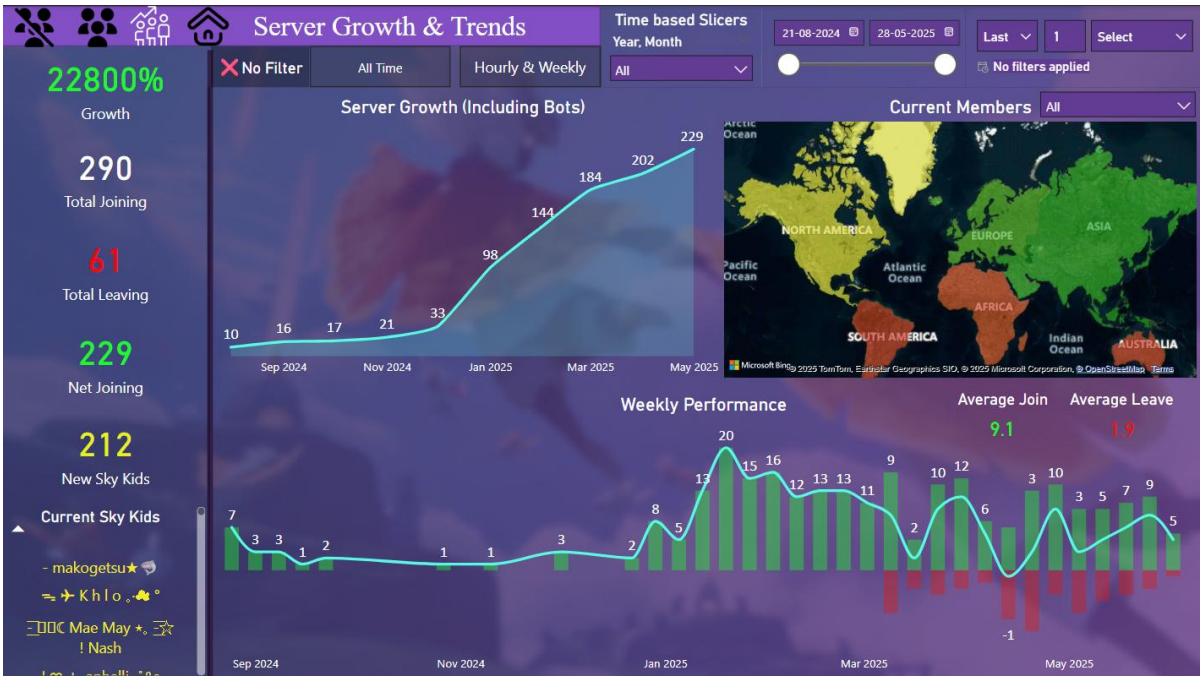



When we open the dashboard, Server Status page is our **command center**.

At the top, summary tiles show the current member count, number of bots, info about roles and a donut chart that breaks down member availability. While the below section shows the members who joined or left in the recent **7 days**. Gauges and KPIs display net joining, growth rate, retention, and churn - giving you a precise pulse on **recent performance**.

In seconds, I know if the server is healthy, who's new and if there are any warning sings.

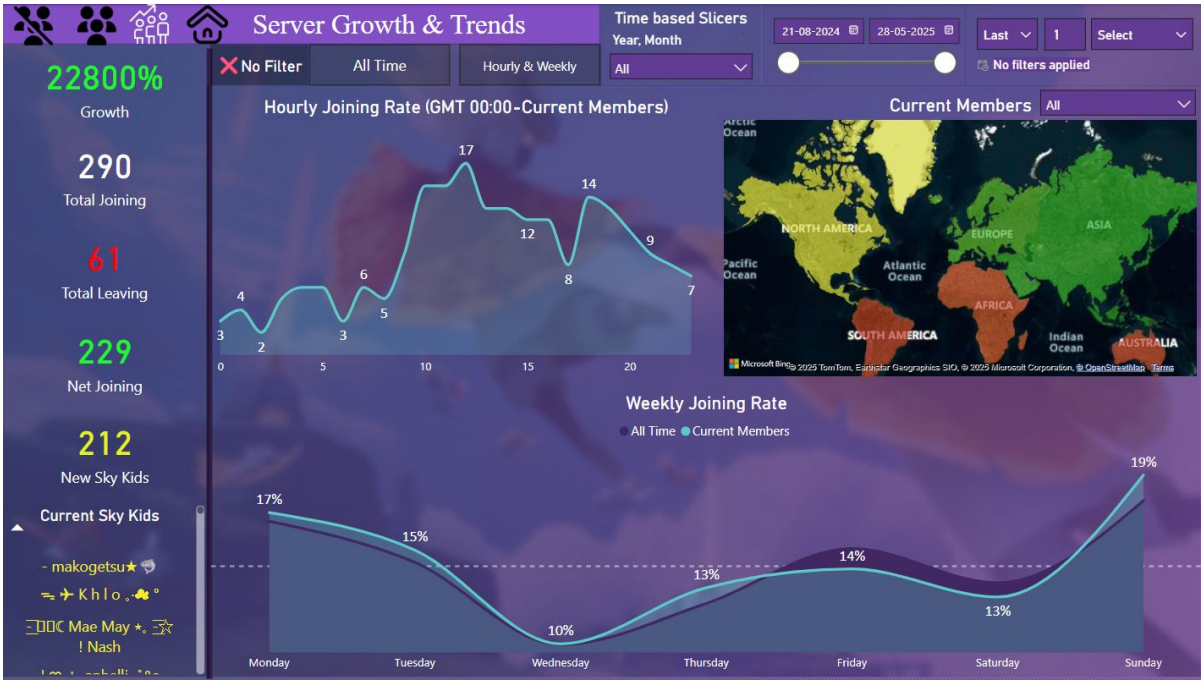
**Growth Insight:** Here, line chart shows the running total of members, while bar charts break down weekly joining and leaving trends to know about the historic trends and patterns.



On the left-hand side, several metrics have been added to know about the number of people who joined or left in a particular timeframe with the growth rate as well.

A map chart has been added to know where the growth is actually coming from.

A bookmark has been created to look at hourly and weekly based trends and patterns too.




To make it more dynamic, time-based slicers have also been added to know about monthly or weekly trends and pattern.

This page helps us to answer questions like: When did we grow faster? Are we losing momentum? Is our outreach reaching new continents? Etc.

**Member Insights:** This page is all about the people who make this server special.

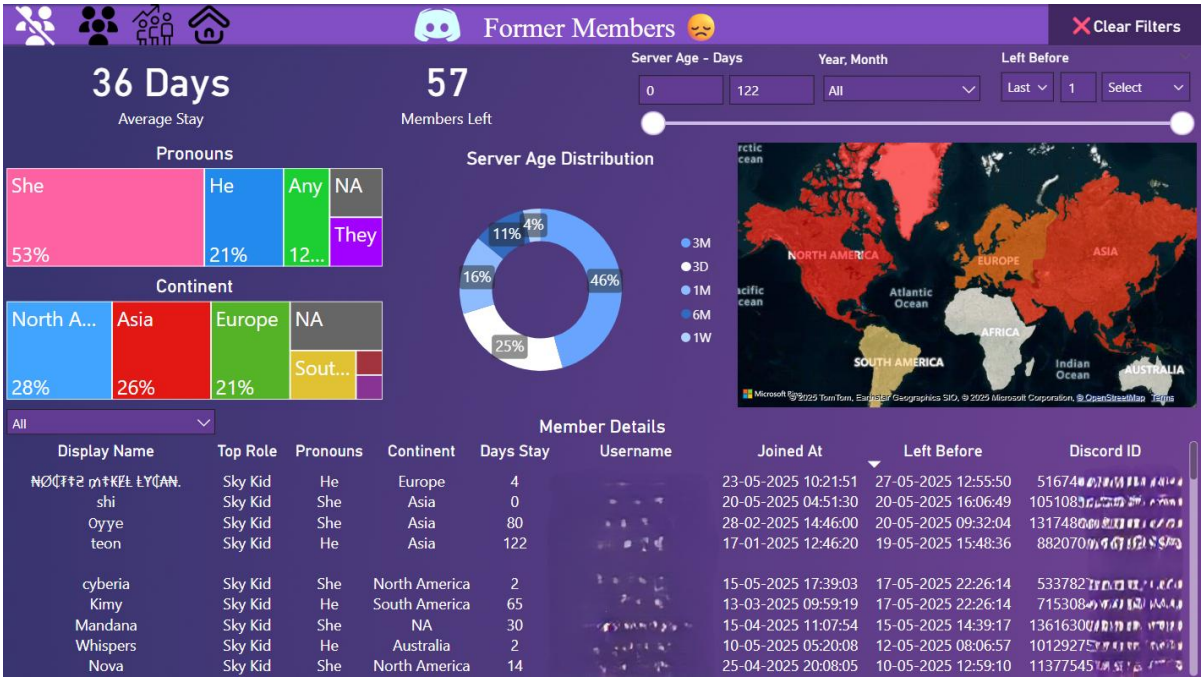


The highlights of this page are:

- It shows the clear distribution of activity-based roles, pronouns, continent, server age and top roles.
- A map chart, to easily know about the continent they are from.
- A detailed and easily comprehensible table list with essential details about each member.
- Through the slicers we can find out the boosters and current status of members.

Understanding the audience helps me tailor events, content and communication-making everyone feel seen and valued.

**Former Member Insights:** Every community has members who leave. The Former Members page helps you learn from them.




At the top, you'll see the average stay (e.g., 36 days) and total members lost. Treemaps show pronouns and continent distribution, while donut chart reveals how long people stayed before leaving. An interactive map visualizes where former members are from, and slicers let you filter by time or server age.

A detailed table lists each former member's journey-roles, pronouns, continent, join/leave dates, and more. This helps to know if members from a certain region are leaving more, or if churn is highest in the first week.

By spotting patterns in departures, we can improve onboarding, target at-risk groups, and boost retention-turning every loss into a lesson.

Now, with a single glance, I can see how our server is doing, who's joining or leaving, and what makes our members unique. We can spot trends, celebrate wins, and address challenges before they become problems.

If you want to truly understand and grow your Discord community, an interactive dashboard like this is your secret weapon. It turns raw data into living story, one that helps you build a stronger, more welcoming and more successful server for everyone.

# Insights

**Performance in the Last 7 Days**

As of **May 28, 2025**, our Discord server has grown to **213 members**, including **17 bots**.The community is organized around **4 top roles** and **15 interest**- based roles, reflecting a diverse range of activities and interests. Over the past week, the server experienced a healthy **5% growth rate**, with a net gain of **11 members**. Most impressively, our retention rate stands at **99.54%,** while the churn rate is just **0.5%.**

This means nearly all new members are choosing to stay, a strong indicator of a welcoming and engaging environment.

**Server Growth Insights**

Looking at the bigger picture, the server's lifetime growth is extraordinary, an increase of **22,700%** from just a single member to over **225 months in 8 months**. A significant growth spike occurred after the server was listed on the promotion platform Disboard in **January**, resulting in a 3x increase and 65 new members that month.

However, after this surge, growth rates have gradually declined: **45% February, 27% in March**, and just **9% in April**. The majority of current members joined between **GMT 09:00 and GMT 21:00**, with over **60% hailing from Europe and Asia**. On average, about **9 new members join and 2 leave each week**, indicating steady, if slowing, growth. One more noticeable pattern is that, now that we have officially added Contest and gift-based roles, less members are leaving the server, indicating **reward-based retention**.

**Member Insights**

Our community is notably **diverse**: More than half of the members identify as female, and **57%** are from Europe and Asia. While over **55%** express interest in fun night activities, actual participation in weekly fun nights remain low (3-7 attendees), suggesting a **disconnect between stated interests and event engagement**. Possible due to timing or activity selection. Additionally, more than half of our current members have joined within the last three months, pointing to a **young and dynamic community**.

**Former Member Insights:**

Former members typically stayed for about **36 days** before leaving. Interestingly, **28%** of those who left were from **North America**, hinting that time zone differences or scheduling may be a factor. A striking 90% of departures occurred within **3 months** of joining, highlighting the importance of early engagement and onboarding.

**Availability Insights:**

By observing when members are most active and available, we can better schedule events and announcements to maximize participation and engagement.

Actionable Steps:

- Optimize Event Timing, especially to entertain members from different regions. like assigning North American based mod to host events and gather feedback.
- Enhance onboarding and early engagement experience, to improve retention in first crucial months.
- As more than 50% of members are female, make sure to prioritize safe and respectful environment comfortable for females.
- Keep leveraging reward-based roles to increase participation in fun nights and events.
- Continue leveraging analytics to monitor trends, test new strategies, and adapt quickly
- Solicit direct feedback from members to supplement analytics and understand the "why" behind the numbers.

The server is thriving, with strong retention and welcoming atmosphere. To sustain growth and deepen engagement, we need to focus on early member experience, optimize event timing, and keep using data to guide our decisions. This approach will help to build a vibrant, resilient Discord community for the long term.

# Learnings

This project turned out to be more than just a technical experiment. It became a deep dive into how data can help us understand and improve online communities. Here are some of the key things I learned during the process:

### Building End-to-End Data Pipeline:

One of the most valuable lessons was understanding how to connect different tools to build a seamless pipeline. From writing a customer Python bot to scrap Discord server data, to storing the data in PostgreSQL Database and finally bringing it to life through interactive dashboards in Power BI. Every step taught me how to move and transform data with purpose.

The entire pipeline became something I could run with just a click, thanks to a bit of batch scripting magic. No more repetitive manual steps. Just clean structured data and up-to-date visuals, ready to go.

### From Raw Data to Meaningful Insights

Before this project, I used to look at numbers as something static. Now, I see them as a story waiting to be told.

By creating calculated measures like retention rate, churn, and growth trends in Power BI, I learned how to turn server activity into insights that actually mean something. Like figuring out when people usually join, which roles attract the most interest, or how event timing affects engagement.

---

*It's one thing to gather data, but making it actionable is where the real value lies.*

---

### Solving Real Problems with Data

This wasn't just about playing with tools. It was about answering real questions:

- Why are people leaving the server?
- What makes them stay longer?
- Are we actually growing, or just adding and losing members at the same pace?

Solving these challenges taught me how to look beyond surface-level stats and dig into patterns, behaviors, and even time zones that affect how people interact online.

### Leveraging A.I.

One of the most interesting parts of this project was how I used AI. Not to do the work for me, but to support and guide me when I needed a second opinion or got stuck.

Throughout the project, I turned to tools like ChatGPT to:

- Get ideas on how to structure my Python Logic more efficiently
- Explore different ways to design database schemas for better performance.
- Understand and fix tricky bugs, especially around Discord's API behavior.
- Think through edge cases like how to handle members rejoining the server.
- Simplify repetitive tasks by improving batch file automation.

Again, to be clear, I wrote the code, made the decisions, and built the system myself. AI was like a helpful teammate: it didn't hand me answers, but it helped me think more clearly, explore alternatives, and spot potential issues I hadn't considered.

It was a reminder that AI is tool, not a shortcut. When used thoughtfully, it can expand your thinking, but the real progress still comes from our own understanding curiosity, and problem-solving.

## Limitations

Though this project has laid a strong foundation for data-drive server management, it's important to acknowledge some current limitations that could be addressed in future updates:

- **Manual Execution:** Right now, the script requires manual execution to refresh data. Means updates aren't happening in real time which can cause the Insights to lag behind actual server activity.
- **Limited Time Granularity:** Activity and status data are captured only during script runs which limits the real time trend detection.
- **Local Dependency:** Since the system runs locally, there is a dependency on machines availability. The data won't update unless I'm physically present near my laptop.
- **No Predictive Features:** While the dashboard shows historical trends well, it doesn't predict user churn and forecast engagement or growth.

Overall, these limitations don't take away from the project's value. They just highlight how much potential is left to explore. Each one is a stepping stone for the next version.

# Security Measures

Discord's API is designed with privacy in mind. It **does not expose sensitive personal information** like email address, phone number, or private messages.

The data collected (username, join dates, roles, etc.) is publicly visible on the server and used solely for community analytics.

- **Hidden Usernames & IDs:** Although Discord IDs and usernames are technically available, I've made sure to mask or exclude them from all dashboard and public views. This ensures to keep the identities of individuals remain private and no one can trace visuals back to specific users.
- **Environment Variables for Credentials:** To protect access credentials like the bot token and database password, I used .env files. So, nothing sensitive is hardcoded into the script. This reduces the risk of accidental leaks or misuse, especially if the code is ever shared or uploaded online.
- **Local Execution for Enhanced Control:** As scripts are run locally on my system, not on a cloud platform. This offers reduced exposure to online security breaches or unauthorized access.

These measures ensure that the project stays focused on its purpose: helping server admins make informed decisions using data, without compromising the privacy or trust of community members.

# Future Scope & Opportunities

The current setup already gives powerful insights, but the possibilities go far beyond. With cloud services and AI tools, this project can evolve into a smart, scalable system.

- **Read-Time Sync with Cloud Scheduling** using platforms like Google Cloud Functions to schedule data extraction script to run at set intervals and updates the database automatically, no local setup needed.
- **Track Member Activity by Region & Time** by logging users availability status through the day using the bot, and store this in a time-series database like Google BigQuery or Amazon Timestream.
  For example, finding that most user from Asia are online at 4PM GMT can help mods to schedule events accordingly.
- **Sentiment Analysis via AI Models** by using APIs like OpenAI, Google Cloud Natural Language to analyze messages for emotional tone. (positive, negative, neutral).
  For example, if sentiment drops in a particular channel (e.g., lots of frustration words), we can step in early to support the members or tweak our moderation approach.
- **Predictive Analysis using AI models** (like decision trees or logistic regressions) trained on historical data to predict things like churn or inactivity.
- **Server Health Score Metric** by combining multiple metrics like growth, sentiment, activity, retention, etc., calculated with DAX or in a cloud function.
  For example, dashboard shows a health score of say, 87/100 (High sentiment, good retention, but a drop in event participation).
- **Customized Insights Access**: We can host the dashboard on Power BI Service or Google Data Studio with role-based access, so different team members see only the insights relevant to them. Like, moderators can view sentiment and churn insights, while organizers see peak hours and interest trends.

Cloud systems and AI don't just add flash—they scale potential. They make projects like this always-on, intelligent, and ready to grow. The foundation I've built can be upgraded to support real-time analytics, sentiment tracking, and predictive modeling. And what's most exciting? Many of the tools needed are accessible, free, or low-cost—even for a solo creator.

# Conclusion

What began as a simple curiosity – "How is our Discord server doing?", evolved into a complete data analytics system that blends essential data analytics tools (Python, SQL, Power BI), automation and community insight into one powerful tool. Through this project, I didn't just build a dashboard. I built an end-to-end data pipeline that collects, stores, analyzes, and visualizes real-time community data.

*This project wasn't built in one go.*

There were bugs, re-thinks, moments of "Why is this not working?!", and lots of trial and errors.

But breaking everything down into phases helped me stay focused and make steady progress.

More importantly, I learned that data isn't just technical. It's personal. Behind every number is a user, a behavior, a moment.

*When u build with empathy and curiosity, even the smallest dataset can tell a powerful story.*

Along the way, I discovered something even more important than technical skill: **Empathy**. Behind every data point is a person—a behavior, an emotion, a story.

This project showed me that data isn't just technical—**it's human**. And when approached with purpose, it can spark real insight, drive engagement, and strengthen community bonds.

There's still so much more to explore and I'm excited to continue improving and expanding this system in the future. But for now, I'm incredibly proud of what this project has become: A simple dashboard, powered by code, driven by data, and built with heart.

**Thank you for reading, and I hope this inspires others to turn data into something truly meaningful.**